

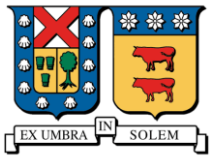
ICN292-Lab 5: SQL (2/2)

Tomás Sánchez Ferrari

Profesor: Patricio Cifuentes

Paralelo 100

URL GitHub: https://github.com/tomassanchez2004/ICN292-Lab5-Sanchez-Tomas



Consulta 1

Código SQL:

```
SELECT ve.nombre,
    COUNT(v.id_venta) AS cantidad_ventas
FROM vendedores ve
LEFT JOIN ventas v
    ON ve.id_vendedor = v.id_vendedor
GROUP BY ve.id_vendedor, ve.nombre
ORDER BY cantidad_ventas DESC, ve.nombre ASC;
```

```
75 •  SELECT ve.nombre,
76        COUNT(v.id_venta) AS cantidad_ventas
77    FROM vendedores ve
78    LEFT JOIN ventas v
79        ON ve.id_vendedor = v.id_vendedor
80    GROUP BY ve.id_vendedor, ve.nombre
81    ORDER BY cantidad_ventas DESC, ve.nombre ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

nombre	cantidad_ventas
Camila Ortiz	4
Diego Fuentes	3
Ignacio Vera	3
Javiera Soto	3
Paula Riquelme	3
Matías Herrera	2
Fernanda Lillo	0
Rubén Castro	0

Consulta 2

Código SQL:

```
SELECT t.nombre AS tienda,
    COUNT(v.id_venta) AS cantidad_ventas,
    COALESCE(SUM(v.unidades), 0) AS unidades_vendidas
FROM tiendas t
LEFT JOIN vendedores ve
    ON t.id_tienda = ve.id_tienda
LEFT JOIN ventas v
    ON ve.id_vendedor = v.id_vendedor
GROUP BY t.id_tienda, t.nombre
ORDER BY unidades_vendidas DESC;
```

```
82 •  SELECT t.nombre AS tienda,
83        COUNT(v.id_venta) AS cantidad_ventas,
84        COALESCE(SUM(v.unidades), 0) AS unidades_vendidas
85    FROM tiendas t
86    LEFT JOIN vendedores ve
87        ON t.id_tienda = ve.id_tienda
88    LEFT JOIN ventas v
89        ON ve.id_vendedor = v.id_vendedor
90    GROUP BY t.id_tienda, t.nombre
91    ORDER BY unidades_vendidas DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

tienda	cantidad_ventas	unidades_vendidas
Andes Concepción	6	14
Andes Providencia	7	11
Andes Maipú	5	10
Andes La Serena	0	0

Consulta 3

Código SQL:

```
SELECT
    t.nombre AS tienda
FROM tiendas t
LEFT JOIN vendedores ve
    ON t.id_tienda = ve.id_tienda
WHERE ve.id_vendedor IS NULL
```

```
92 •  SELECT
93        t.nombre AS tienda
94    FROM tiendas t
95    LEFT JOIN vendedores ve
96        ON t.id_tienda = ve.id_tienda
97    WHERE ve.id_vendedor IS NULL;
```

Result Grid | Filter Rows: | Export:

tienda
Andes La Serena

Consulta 4

Código SQL:

SELECT b.modelo

FROM bicicletas b

LEFT JOIN ventas v

ON b.id_bicicleta = v.id_bicicleta

AND v.fecha >= '2025-06-01'

AND v.fecha < '2025-07-01'

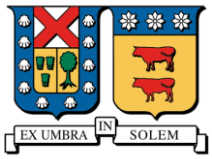
WHERE v.id_venta IS NULL;

```
98 •  SELECT b.modelo
99    FROM bicicletas b
100   LEFT JOIN ventas v
101       ON b.id_bicicleta = v.id_bicicleta
102       AND v.fecha >= '2025-06-01'
103       AND v.fecha < '2025-07-01'
104   WHERE v.id_venta IS NULL;
```

Result Grid | Filter Rows: | Export:

modelo
Ruta Austral
Urbana Pro
Volt City
Sprint R7

La condición del mes se ubicó en el "ON", si se hubiera colocado en el "WHERE", solo aparecerían los modelos que sí registraron ventas en junio de 2025.



Consulta 5

Código SQL:

```
SELECT ve.nombre AS vendedor,
    COALESCE(t.nombre, 'Sin tienda asignada') AS tienda
FROM vendedores ve
LEFT JOIN ventas v
    ON ve.id_vendedor = v.id_vendedor
LEFT JOIN tiendas t
    ON ve.id_tienda = t.id_tienda
WHERE v.id_venta IS NULL;
```

```
105 •  SELECT ve.nombre AS vendedor,
106        COALESCE(t.nombre, 'Sin tienda asignada') AS tienda
107    FROM vendedores ve
108    LEFT JOIN ventas v
109        ON ve.id_vendedor = v.id_vendedor
110    LEFT JOIN tiendas t
111        ON ve.id_tienda = t.id_tienda
112    WHERE v.id_venta IS NULL;
```

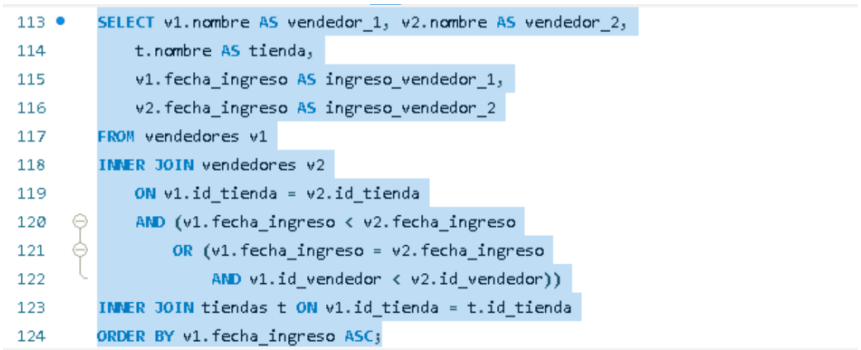
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

vendedor	tienda
Fernanda Lillo	Andes Providencia
Rubén Castro	Sin tienda asignada

Consulta 6

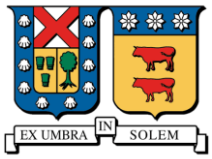
Código SQL:

```
SELECT v1.nombre AS vendedor_1, v2.nombre AS vendedor_2,
    t.nombre AS tienda,
    v1.fecha_ingreso AS ingreso_vendedor_1,
    v2.fecha_ingreso AS ingreso_vendedor_2
FROM vendedores v1
INNER JOIN vendedores v2
    ON v1.id_tienda = v2.id_tienda
    AND (v1.fecha_ingreso < v2.fecha_ingreso
        OR (v1.fecha_ingreso = v2.fecha_ingreso
            AND v1.id_vendedor < v2.id_vendedor))
INNER JOIN tiendas t ON v1.id_tienda = t.id_tienda
ORDER BY v1.fecha_ingreso ASC;
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

vendedor_1	vendedor_2	tienda	ingreso_vendedor_1	ingreso_vendedor_2
Paula Riquelme	Ignacio Vera	Andes Concepción	2022-11-02	2024-08-19
Javiera Soto	Matías Herrera	Andes Maipú	2023-07-24	2024-01-15
Diego Fuentes	Camila Ortiz	Andes Providencia	2024-01-15	2024-06-20
Diego Fuentes	Fernanda Lillo	Andes Providencia	2024-01-15	2025-02-03
Camila Ortiz	Fernanda Lillo	Andes Providencia	2024-06-20	2025-02-03



Consulta 7

Código SQL:

```
SELECT t.nombre AS tienda,
    SUM(v.unidades * b.precio) AS monto_total
FROM tiendas t
INNER JOIN vendedores ve ON t.id_tienda = ve.id_tienda
INNER JOIN ventas v ON ve.id_vendedor = v.id_vendedor
INNER JOIN bicicletas b ON v.id_bicicleta = b.id_bicicleta
WHERE b.categoria IN ('Montaña', 'Ruta')
GROUP BY t.id_tienda, t.nombre
HAVING SUM(v.unidades * b.precio) > 2000000
ORDER BY monto_total DESC;
```

```
125 •  SELECT t.nombre AS tienda,
126        SUM(v.unidades * b.precio) AS monto_total
127    FROM tiendas t
128    INNER JOIN vendedores ve ON t.id_tienda = ve.id_tienda
129    INNER JOIN ventas v ON ve.id_vendedor = v.id_vendedor
130    INNER JOIN bicicletas b ON v.id_bicicleta = b.id_bicicleta
131    WHERE b.categoria IN ('Montaña', 'Ruta')
132    GROUP BY t.id_tienda, t.nombre
133    HAVING SUM(v.unidades * b.precio) > 2000000
134    ORDER BY monto_total DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

tienda	monto_total
Andes Providencia	4399930.00
Andes Concepción	3499950.00

Consulta 8

Código SQL:

SELECT modelo, precio

FROM bicicletas

WHERE precio > (SELECT AVG(precio)

FROM bicicletas)

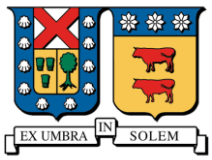
ORDER BY precio DESC;

```
135 •   SELECT modelo, precio
136     FROM bicicletas
137     WHERE precio > (SELECT AVG(precio)
138         FROM bicicletas)
139     ORDER BY precio DESC;
```

Result Grid | Filter Rows: | Export: | Wrap

modelo	precio
Volt City	1249990.00
Sprint R7	1099990.00
Ruta Austral	899990.00
Trail Master	749990.00

El precio promedio del catálogo es de $657.490, calculado por la función AVG() aplicada a la columna precio de la tabla bicicletas y se utilizó una consulta para seleccionar solo los modelos cuyo precio supera el promedio calculado.



Consulta 9

Código SQL:

```
SELECT ve.nombre, AVG(v.unidades) AS promedio_unidades
FROM vendedores ve
INNER JOIN ventas v
    ON ve.id_vendedor = v.id_vendedor
GROUP BY ve.id_vendedor, ve.nombre
HAVING AVG(v.unidades) > (SELECT AVG(unidades)
    FROM ventas)
ORDER BY promedio_unidades DESC, ve.nombre ASC;
```

```sql
SELECT ve.nombre, AVG(v.unidades) AS promedio_unidades
FROM vendedores ve
INNER JOIN ventas v
    ON ve.id_vendedor = v.id_vendedor
GROUP BY ve.id_vendedor, ve.nombre
HAVING AVG(v.unidades) > (SELECT AVG(unidades)
    FROM ventas)
ORDER BY promedio_unidades DESC, ve.nombre ASC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nombre	promedio_unidades
Paula Riquelme	2.6667
Javiera Soto	2.3333
Diego Fuentes	2.0000
Ignacio Vera	2.0000



Consulta 10

Código SQL:

SELECT ve.nombre,

*SUM(v.unidades * b.precio) AS monto_total*

FROM vendedores ve

LEFT JOIN ventas v ON ve.id_vendedor = v.id_vendedor

LEFT JOIN bicicletas b ON v.id_bicicleta = b.id_bicicleta

GROUP BY ve.id_vendedor, ve.nombre

ORDER BY monto_total DESC;

```sql
SELECT ve.nombre,
    SUM(v.unidades * b.precio) AS monto_total
FROM vendedores ve
LEFT JOIN ventas v ON ve.id_vendedor = v.id_vendedor
LEFT JOIN bicicletas b ON v.id_bicicleta = b.id_bicicleta
GROUP BY ve.id_vendedor, ve.nombre
ORDER BY monto_total DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

nombre	monto_total
Camila Ortiz	3429950.00
Paula Riquelme	3149920.00
Matías Herrera	3049970.00
Diego Fuentes	3009940.00
Ignacio Vera	2319940.00
Javiera Soto	2009930.00
Fernanda Lillo	NULL
Rubén Castro	NULL

Los vendedores que no registraron ventas muestran NULL porque no existen registros asociados para calcular su monto total, para mostrar cero se debe agregar COALESCE(SUM(v.unidades * b.precio), 0).